



INSTITUTO FEDERAL DO RIO GRANDE DO NORTE – CAMPUS CAICÓ

CURSO TÉCNICO INTEGRADO EM INFORMÁTICA PARA INTERNET

Projeto de Interface de Usuário

Prof. Ciro Medeiros

10/04/2026

A7 – JavaScript: async/await

Instruções

1. Esta lista é essencialmente uma recriação da lista de exercícios A6. Salvo em exceções explícitas, procure reescrever as respostas usando `async/await` e a estrutura `try-catch` comum em vez de `Promises` explícitas e seus métodos `.then` e `.catch`.
2. Todas as questões devem ser respondidas em código JavaScript.
3. **Não faça entrada de dados**, basta criar variáveis com valores pré-definidos para testar.
4. Essa lista serve para treinar sintaxe JavaScript, então algumas questões pedem construções específicas. Preste atenção ao enunciado.
5. Responda com cada questão em um arquivo separado (q1.js, q2.js, q3.js, ...). **Não mande em arquivo compactado.**
6. Cada questão vale 1 ponto. Total de **8 pontos**.

Questões

Q1 – Crie uma função `async getStatus(condicao)` que recebe um booleano. Se `true`, retorna `"Sucesso!"`. Se `false`, lança erro usando `throw` com a mensagem `"Falha!"`. Consuma a função dentro de um bloco `try-catch`.

Q2 – Crie uma função `async verificarPositivo(n)` que:

1. Se `n` não for um `number`, lança um erro `"Tipo XX não é number."`.
2. Se `n` for um `number`, mas for menor que 0, lança um erro `"O valor XX deveria ser maior que 0"`.
3. Se `n` for um `number` positivo, não faz nada, apenas retorna.

Na chamada da função no programa principal, exiba “Número XX é positivo.” em caso de sucesso dentro do bloco `try` ou a mensagem retornada em caso de erro dentro do bloco `catch`.

Q3 (Esta questão exige o uso da `Promise` explicitamente) – Crie uma função `esperar(ms)` que recebe um tempo em milissegundos e retorna uma `Promise`. A `Promise` retornada espera o dado tempo e resolve passando o valor “`Terminou!`”. No programa principal, antes de chamar a função `esperar`, exiba “`Esperando XXXXms...`” no console. Em seguida, chame a função `esperar` com `await` para exibir a mensagem no console. O que acontece se você retirar o `await`? Experimente.

Q4 (Esta questão sugere o uso do `.then` em vez de `try-catch`) – Crie uma função `async` que retorna o número 5. Encadeie dois `.then()`:

- Primeiro: recebe o número e retorna seu dobro.
- Segundo: recebe o resultado anterior e retorna seu triplo.

Exiba o resultado final (deve ser 30).

Q5 – Reutilize a função da questão Q1. Crie uma lista com 3 chamadas assíncronas a `getStatus` passando `true` em todas e execute com `await Promise.all`. O que acontece se você passar `false` para uma delas? Experimente.

Q6 – Reutilize o código da questão Q5, porém agora execute a lista de chamadas assíncronas com `await Promise.allSettled`. O que acontece agora se passar `false` para uma delas? Experimente.

Q7 – Adapte o código da questão Q6 de maneira que a função `async getStatus` receba um `id` e um booleano `condicao` e:

1. Espere um tempo aleatório entre 1 e 3 segundos.
2. Retorne “`(id) Sucesso :`” ou lance erro “`(id) Erro :(`” de acordo com o booleano informado.

Faça 3 chamadas assíncronas (`ids` 1, 2 e 3) e execute com `await Promise.any`. Experimente colocar todos os valores booleanos `true`, todos `false` e misturá-los. Analise o resultado.

Q8 – Repita o código da questão Q7, porém execute com `await Promise.race`. Analise o resultado.